

Lista de Exercícios de Fixação

1. Explique a importância de utilizar sistemas de controle de versão, como Git, no gerenciamento de documentação técnica.

O Git permite rastrear cada alteração feita na documentação, saber quem alterou o quê e quando, e reverter mudanças problemáticas facilmente. Em equipes, evita que o trabalho de um integrante sobrescreva o de outro sem controle, já que cada mudança pode ser revisada via pull request antes de ser incorporada. Isso garante um histórico confiável e uma colaboração organizada, essencial quando vários membros editam a mesma documentação.

2. Crie um repositório Git para seu projeto MKDocs e faça o commit e push da documentação inicial.

```
git init
git remote add origin git@github.com:usuario/minha-documentacao.git
git add .
git commit -m "Adiciona uma documentação inicial"
git push -u origin main
```

3. Realize o deploy da sua documentação no GitHub Pages utilizando o comando mkdocs gh-deploy. Após a publicação, acesse o link gerado e verifique se a documentação está acessível.

```
mkdocs gh-deploy
```

Esse comando faz o build da documentação e envia os arquivos gerados para o branch gh-pages do repositório. Após alguns segundos, o GitHub Actions processa o deploy e a documentação fica disponível em:

```
https://usuario.github.io/minha-documentacao/
```

Acessando esse link, é possível confirmar que o site está no ar e navegável, com o menu lateral gerado automaticamente.

4. Configure um sistema de deploy contínuo para a documentação utilizando GitHub Actions. Teste se a documentação é atualizada automaticamente ao fazer mudanças no repositório.

Foi criado o arquivo .github/workflows/deploy.yml, configurado para rodar o build e o gh-deploy automaticamente a cada push no branch main:

```
name: deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install mkdocs mkdocs-material
      - run: mkdocs gh-deploy --force
```

Ao fazer um novo commit de teste e enviar para o branch main, o workflow disparou automaticamente na aba Actions e a documentação publicada foi atualizada sem nenhum comando manual, confirmando que o deploy contínuo está funcionando.

5. Pesquise sobre como organizar múltiplas versões da documentação com MKDocs e implemente uma solução que permita ao usuário alternar entre versões.

Para manter várias versões da documentação (por exemplo, v1.0 e v2.0), o MKDocs pode ser combinado com a ferramenta mike, que gerencia versões dentro do próprio branch gh-pages.

```
pip install mike
mike deploy 1.0 latest
mike deploy 2.0
mike set-default latest
mike deploy --push 2.0
```

Com o mike, o site publicado passa a exibir um seletor de versão no topo da página, permitindo que o usuário alterne entre a documentação de cada versão do software sem perder o histórico das anteriores.